

myRpackages

Simon Labarthe

simon.labarthe@outlook.com

Personal Website

GitHub: SimonLabarthe

August 2026

Abstract

This article introduces **myRpackages**, a curated collection of R functions designed to enhance efficiency in data analysis, automated reporting, and AI-supported text classification. Researchers frequently face repetitive tasks in data preparation and output generation. The package addresses this by offering straightforward functions for creating publication-ready LaTeX/PDF tables and integrating with AI APIs for content coding. The core functions, `create_descriptive_table()`, `create_frequency_table_2vars()`, and `categorise_api()`, are presented with practical examples obviously coming from *starwars*. By automating these common analytical workflows, **myRpackages** allows researchers to focus on substantive interpretation rather than technical implementation. The package is open-source and available on GitHub.

Keywords. R package, Data Analysis, Automated Reporting, LaTeX, Text Classification, DeepSeek API.

1 Introduction

Social scientists spend a substantial portion of their time not on thinking, but on the mechanical aspects of data analysis: cleaning data, writing repetitive code to generate tables, and manually classifying text data. This is an inefficient use of intellectual resources (already limited in my case). While R offers powerful analytical capabilities, the path from a clean dataset to a polished, publication-ready table is often fraught with cumbersome code and debugging.

This inefficiency matters because it reduces the time available for theoretical reflection, robust research design, and careful interpretation of results—the core activities that advance scientific knowledge. For political science and other social sciences, tools that expedite the ‘last mile’ of empirical analysis are essential for accelerating the research cycle and improving reproducibility. But how can a set of focused R functions be designed to automate and simplify common yet critical tasks in quantitative data analysis and AI-assisted text coding?

The R ecosystem is rich with powerful packages (e.g., `dplyr` for data manipulation, `xtable` and `stargazer` for table generation) that are highly flexible but often require verbose syntax. This flexibility can be a barrier for efficient workflows. The package presented here fills a gap by providing a suite of high-level, opinionated functions that execute these tasks with a single line of code, while generating outputs in a polished, publication-ready format. This article addresses this gap by introducing a cohesive toolkit designed for speed and simplicity.

`myRpackages` offers a pragmatic solution by wrapping complex R operations into intuitive functions that produce LaTeX and PDF outputs for descriptive and frequency tables, and seamlessly integrate with the DeepSeek API for AI-powered text categorisation.

I contribute to the community by providing a lightweight, open-source R toolkit that lowers the technical overhead for creating high-quality analytical outputs. It is a practical resource for researchers seeking to streamline their empirical workflow.

Section 2 provides a comprehensive overview of the included functions. Section 3 details the design and implementation of a key function, `categorise_api()`, used for AI-powered text analysis. Section 4 demonstrates the package’s utility through a practical research workflow example. Section 5 concludes with a discussion of limitations and future developments.

2 Package Overview and Core Functions

`myRpackages` is a collection of R utility functions designed for data analysis and automated reporting. The package is designed to be intuitive, requiring minimal R expertise to generate high-quality outputs.

2.1 Installation and Quick Start

The package can be loaded directly from GitHub using the provided script. To get started, run the following in your R console:

```
1 # Load and install all functions with one line
2 source("https://raw.githubusercontent.com/SimonLabarthe/
   myRpackages/main/load_all.R")
3 load_myRpackages()
```

Listing 1: Loading `myRpackages`

This command sources and loads the core functions into the global environment. For PDF and LaTeX output, a LaTeX distribution is required, which can be conveniently installed via R:

```
1 install.packages("tinytex")
2 tinytex::install_tinytex()
```

Listing 2: Installing LaTeX support

2.2 Available Functions

The current version of the package (August 2026) includes the following key functions:

1. `create_descriptive_table()`: Generates a comprehensive descriptive statistics table (including n, mean, median, mode, SD, min, max, and quartiles) for a given dataset. It creates a LaTeX /.tex file and compiles it into a standalone PDF, offering extensive customization options (e.g., selecting which statistics to display).
2. `create_frequency_table_2vars()`: Creates a cross-tabulation frequency table for two variables, outputting results in a similar polished PDF format.
3. `create_frequency_table_3vars()`: Extends the frequency table functionality to three variables, producing a nested frequency table that is particularly useful for subgroup analysis.
4. `categorise_api()`: An advanced function that uses the DeepSeek API to classify a vector of texts. It allows researchers to efficiently code large text corpora based on a custom taxonomy, with built-in checkpointing and batch processing.

2.3 Design Philosophy

The design philosophy behind `myRpackages` is guided by two key principles. First, **laziness**: Each function is designed to perform a complete task with minimal user inputs. The goal is to reduce complex data processing and output generation to a single, readable line of code. For example, generating a full descriptive statistics table in PDF is as simple as:

```
1 create_descriptive_table(data, output_dir = getwd())
```

Second, **maximum coolness**: The emphasis on LaTeX and PDF generation ensures that the output is ready for immediate integration into manuscripts, reports, or presentations. This approach bridges the gap between data analysis and scholarly publication.

3 Use Case: AI-Powered Text Categorisation with `categorise_api`

A significant challenge in social science research is the efficient and reliable coding of textual data. The `categorise_api()` function directly addresses this challenge by leveraging a Large Language Model (LLM) through an API. This section details its implementation and utility.

3.1 Function Overview and Implementation

The `categorise_api()` function takes a character vector of texts and uses DeepSeek's API to classify them into predefined categories and themes. Its design incorporates several key features to ensure robustness and efficiency for large-scale projects:

- **Batch Processing:** To manage API costs and handle rate limits, texts are processed in batches (default size is 20). This prevents server timeouts and allows for efficient processing of large datasets.
- **Checkpointing:** This is a critical feature for long-running tasks. The function regularly saves progress to a local RDS file (e.g., `checkpoint_categorise.rds`). If the process is interrupted, it can seamlessly resume from the last saved checkpoint, avoiding the need to reprocess already-coded texts.
- **Structured Prompting:** The function uses a carefully designed prompt that instructs the AI to provide responses as a JSON array, ensuring the output is structured and easily parsed. The prompt defines specific thematic taxonomies (e.g., 'care', 'feminist interests') with clear sub-categories (e.g., 'childcare', 'gender equality'), enabling detailed and consistent coding.
- **Error Handling and Retry Logic:** The loop includes `tryCatch()` to handle potential API errors. If a batch fails, it logs an error message and continues to the next batch without crashing the entire process, which is crucial for unsupervised automation.

3.2 Practical Example

The following R code demonstrates how to use `categorise_api()` to classify a set of political texts.

```
1 # Load the package
2 source("https://raw.githubusercontent.com/SimonLabarthe/
   myRpackages/main/load_all.R")
3 load_myRpackages()
4
5 # Sample texts
6 my_texts <- c(
7   "The new childcare policy will reduce costs for
   families.",
8   "We must fight for equal pay for equal work.",
9   "The traditional family unit is the foundation of our
   society."
10 )
11
12 # Set your API key (ideally stored in .Renviron)
13 api_key <- Sys.getenv("DEEPSEEK_API_KEY")
14
15 # Categorise the texts
16 results <- categorise_api(texts = my_texts,
17                           api_key = api_key,
18                           batch_size = 2,
19                           checkpoint_file = "my_
   categorisation.rds")
20
21 # View the results
22 print(results)
```

Listing 3: Classifying texts with `categorise_api()`

3.3 Expected Output

The function would return a data frame with the same number of rows as the input texts, containing the predicted ‘category’ and ‘theme’ for each text.

Table 1: Example Output from `categorise_api()`

Text	Category	Theme
The new childcare policy...	care	childcare
We must fight for equal pay...	feminist interests	pay gap
The traditional family unit...	gendered interests	traditional family

3.4 Recommendations

This function is particularly useful for exploratory research, large-N content analysis, and as a tool to pre-code qualitative data for mixed-methods studies. The built-in checkpointing makes it ideal for processing large corpora where API calls may be costly or prone to failure.

4 A Practical Research Workflow

This section provides a step-by-step example of how `myRpackages` can be integrated into a typical research project workflow, demonstrating its interoperability with common R packages.

4.1 Step 1: Environment Setup and Data Import

First, load the package and import your dataset. The functions are designed to work seamlessly with standard R data frames (e.g., from `read.csv()` or `readRDS()`).

```
1 # Load myRpackages
2 source("https://raw.githubusercontent.com/SimonLabarthe/
  myRpackages/main/load_all.R")
3 load_myRpackages()
4
5 # Load necessary libraries
```

```

6 library(dplyr)
7 library(ggplot2)
8
9 # Load a sample dataset
10 data("mpg")

```

Listing 4: Setting up the environment

4.2 Step 2: Exploratory Analysis and Data Transformation

Use familiar dplyr verbs to prepare your data. For this example, we will create a summary dataset to demonstrate a frequency table.

```

1 # Create a factor variable for drv
2 mpg$drv <- factor(mpg$drv, labels = c("Front-wheel", "
  Rear-wheel", "4-wheel"))
3 # Create a factor for cyl
4 mpg$cyl <- factor(mpg$cyl)

```

Listing 5: Preparing the data

4.3 Step 3: Generating Descriptive and Frequency Tables

Now, apply the core functions of the package to create publication-ready outputs.

```

1 # Generate descriptive statistics for the 'mpg' dataset
2 create_descriptive_table(mpg,
3                           output_name = "mpg_descriptives"
4                           ,
5                           output_dir = getwd(),
6                           mean = TRUE, median = TRUE, sd =
9                           TRUE,
10                          min = TRUE, max = TRUE,

```

```

7         mode = TRUE)
8
9 # Create a 2-variable frequency table (cyl vs. drv)
10 create_frequency_table_2vars(mpg, var1 = "cyl", var2 = "
    drv")

```

Listing 6: Generating tables

This code will produce PDF files named `mpg_descriptives.pdf` and `frequency_table_2vars.pdf` in the current working directory, ready for inclusion in a report or manuscript.

4.4 Step 4: Analysis and Categorisation of Text Data

Imagine you have a dataset of political speeches alongside structural variables. You can use `categorise_api()` to code the speeches and then merge the resulting categories back into your primary dataset for further analysis. For instance, you could create a new categorical variable from the `categorise_api()` output and then use it in a regression, easily summarised with `create_descriptive_table()` again. This creates a highly efficient and reproducible workflow from raw text to final statistical output.

5 Discussion and Conclusion

5.1 Summary of the Package

`myRpackages` provides a practical and efficient set of tools to address common bottlenecks in the social science research workflow. By wrapping complex operations in simple, one-line functions, it allows researchers to generate high-quality, publication-ready tables and to leverage state-of-the-art AI models for text analysis with ease. The package's emphasis on LaTeX/PDF output and robust features like checkpointing make it a valuable asset for empirical projects of any scale.

5.2 Interpretation and Utility

The package is particularly useful for researchers who are more focused on substantive questions than on coding. By automating repetitive tasks, it frees up intellectual resources for more critical aspects of the research process, such as interpreting results and developing theoretical arguments. The integration of AI text classification, in particular, opens up new avenues for content analysis and mixed-methods research at scale.

5.3 Limitations

As a work-in-progress, the package has certain limitations. Its current scope is focused on descriptive statistics, frequency tables, and text categorisation. It is not intended to replace more specialized packages for complex statistical modelling (e.g., `lme4`, `fixest`) but rather to complement them. Furthermore, the `categorise_api()` function's performance is dependent on the underlying AI model and API reliability and cost. The output is a tool for analysis, requiring human validation to ensure validity, as AI models can still produce errors or biases.

5.4 Future Research Agenda

Future developments will focus on expanding the collection. Planned additions include functions for creating regression tables and coefficient plots directly from common model outputs. Integration with other LLM providers such as OpenAI's GPT could provide more flexibility. Finally, enhancing the customisation options for frequency tables will be a priority.

5.5 Closing Statement

I believe that `myRpackages` can be a valuable asset for the research community. I invite researchers to use it, contribute to it, and adapt it to their needs. The source code is freely available on GitHub, and I welcome any feedback or contributions.

References

- [1] Labarthe, S. (2026). `myRpackages`. <https://github.com/SimonLabarthe/myRpackages>
- [2] RStudio Team (2020). RStudio: Integrated Development for R. RStudio, PBC, Boston, MA. <http://www.rstudio.com/>.
- [3] Xie, Y. (2019). 'TinyTeX: A lightweight, cross-platform, and easy-to-maintain LaTeX distribution based on TeX Live.' *TUGboat*, 40(1), 30-32.
- [4] DeepSeek AI. (2026). <https://www.deepseek.com/>